

NATURAL INTELLECTS LTD

TicketUG

Payment & Organizer Settlement Architecture

Production Financial Model • Version 1.0 • 5 September 2026

Defines how TicketUG processes customer payments, earns platform revenue, records transactions and supports organizer settlement without unnecessarily making Natural Intellects the custodian of organizer funds.

1. Executive Decision

TicketUG should operate primarily as the **event marketplace and ticketing platform**, while a suitable payment service provider handles regulated payment processing and organizer settlement. Natural Intellects should avoid becoming the default custodian of organizer funds unless a later legal, commercial and licensing review explicitly supports that model.

Recommended structure: Attendee → Payment Provider → Organizer settlement + TicketUG platform fee.

2. Why This Model

- Reduces the amount of organizer money Natural Intellects directly holds or manually distributes.
- Reduces reconciliation, payout, refund and operational complexity.
- Separates TicketUG's software/marketplace role from regulated payment infrastructure.
- Scales better as organizer and transaction volume grows.
- Allows different Uganda-capable payment providers to be integrated later.
- Allows TicketUG to earn revenue per successful transaction.

3. Regulatory Position

This is an engineering/product baseline, not legal advice. Uganda's payment-services environment is regulated. Natural Intellects should confirm the final operating model with qualified Ugandan legal/compliance advisers and the selected provider before accepting live funds.

The implementation agent must not design TicketUG to function as an unlicensed payment service merely because it is technically possible to collect money into a company account and later pay organizers.

4. Commercial Money Flow

Preferred model: a provider-supported marketplace/split-settlement arrangement where the provider processes the attendee payment and routes the appropriate amount to the organizer while accounting for TicketUG's platform fee.

Party	Role	Financial responsibility
Attendee	Buyer	Pays the checkout total.
Organizer	Event seller/merchant	Receives entitled settlement.
TicketUG / Natural Intellects	Platform	Earns configured platform/service fee.
Payment provider	Payment infrastructure	Processes payment and performs supported settlement/refund functions.

5. Example Transaction

Illustrative only — percentages are not final.

Item	Amount
Ticket face value	100,000 UGX
Illustrative TicketUG fee (5%)	5,000 UGX
Illustrative customer total if fee is passed to buyer	105,000 UGX
Organizer entitlement	100,000 UGX
Payment processing fee	Depends on provider/agreement

The platform should support both an included fee and a separately disclosed service fee, subject to the final commercial policy.

6. Fee Model

- Platform fee is configurable server-side, never hard-coded in frontend code.
- Fee rules may later vary by organizer, event, ticket type or agreement.
- Payment-provider fees are modeled separately from TicketUG revenue.
- Refund treatment of TicketUG fees must be explicitly defined.
- Discounts, promotions and taxes/levies, if introduced, are separate financial components.

7. Payment Provider Strategy

Do not hard-wire the TicketUG domain to one provider. Implement a provider abstraction with adapters. This lets TicketUG start with a suitable Uganda-capable provider and add alternatives later.

Capability	Requirement	Reason
UGX collections	Required	Initial market is Uganda.
Mobile money	Strongly preferred	Important for local adoption.
Cards	Preferred	Broader payment coverage.
Verified webhooks	Mandatory	Authoritative payment confirmation.
Idempotency	Mandatory	Prevents duplicate processing.
Refunds	Required before launch if promised	Operational necessity.
Organizer settlement	Strongly preferred	Supports recommended model.
Split/platform fees	Strongly preferred	Supports TicketUG monetization.
Reconciliation	Mandatory	Financial integrity.
Sandbox	Mandatory	Safe testing.

8. Nylopay / Provider Selection

Nylopay can be evaluated as a candidate, but it should not be selected solely because it accepts payments. The provider must satisfy TicketUG's complete operational requirements: collections, webhooks, idempotency, refunds, reconciliation, settlement, platform-fee handling, merchant onboarding and appropriate regulatory/commercial arrangements.

Provider decision gate: verify current Uganda availability, merchant onboarding, settlement mechanics, API/webhooks, fees, refunds and marketplace/split-settlement capabilities before production selection.

9. Payment State Machine

Order: PENDING_PAYMENT → PAYMENT_PROCESSING → PAID → FULFILLED/ISSUED. Exceptional states: PAYMENT_FAILED, EXPIRED, CANCELLED, REFUNDED/PARTIALLY_REFUNDED.

Payment: CREATED → INITIATED → PENDING/PROCESSING → SUCCESS or FAILED/EXPIRED. Refund state is represented separately where appropriate.

- Frontend redirects are informational only.
- Verified provider webhook or trusted reconciliation determines authoritative payment status.
- Payment transitions are idempotent.
- Successful payment cannot issue tickets more than once.
- Failed/expired payment cannot create usable tickets.

10. Checkout Workflow

Attendee selects tickets → server validates availability/pricing → server creates immutable order snapshot → server calculates fees/total → payment request is created → attendee pays → verified webhook arrives → server validates amount/currency/order mapping → payment/order become successful → tickets are issued → notification is queued.

11. Inventory & Concurrency

- PostgreSQL is authoritative for inventory.
- Checkout must prevent overselling under concurrent purchases.
- If temporary reservations are used, they expire and release inventory automatically.
- Payment processing must not permanently reserve inventory without a defined expiry/reconciliation mechanism.
- Quantity limits are server-enforced.

12. Organizer Settlement

Prefer provider-supported settlement. TicketUG should still maintain an internal financial ledger and reconciliation view even when the provider performs actual settlement.

Record	Purpose
Order	What the customer purchased.
Order item	Ticket type, quantity and immutable price snapshot.
Payment	Provider transaction and state.
Fee line	TicketUG/platform fee and basis.
Settlement	Amount due/paid to organizer and provider reference.
Refund	Amount returned and reason/status.
Ledger entry	Auditable financial movement.

13. Reconciliation

- Every provider transaction maps to an internal payment/order ID.
- Provider transaction IDs are unique internally.
- Periodic reconciliation compares provider records with TicketUG records.
- Unmatched transactions are flagged for investigation.
- Manual reconciliation changes require actor, reason and timestamp.
- Ledger/reporting distinguishes gross sales, fees, refunds, settlement and platform revenue.

14. Refunds & Cancellations

- Refund eligibility follows published TicketUG and event policies.
- Refund requests are authorized and audited.
- Provider-supported refund mechanisms are preferred.
- Internal state changes follow reliable provider confirmation.
- Refunded/cancelled tickets are invalid for entry.
- Treatment of TicketUG and provider fees must be explicitly defined before launch.

15. Payment Database Additions

Entity	Purpose / important data
Payment	Order/provider IDs, amount, currency, status, idempotency key, timestamps
PaymentAttempt	Attempts, provider response, retry/reconciliation state
FeeLine	Order, fee type, amount, basis, payer, version
Settlement	Organizer, amount, provider reference, status
Refund	Payment/order, amount, reason, provider refund ID, status
WebhookEvent	Provider, external ID, payload hash, processing state
LedgerEntry	Financial movement, source reference, timestamp
ReconciliationRecord	Period, provider/internal references, discrepancy and resolution

16. Payment Security

- Prefer provider-hosted/tokenized payment collection; do not store raw card data.
- Provider credentials and webhook secrets remain server-side.
- Verify webhook signatures before processing.
- Reject duplicate/replayed webhook events.
- Validate provider amount and currency against the internal order.
- Use idempotency keys for payment creation/retries.
- Do not log secrets, tokens or unnecessary sensitive payment data.
- Restrict and audit manual payment/refund operations.
- Rate-limit payment initiation.

17. API Requirements

Endpoint	Required behavior
POST /checkout/orders	Validate inventory/pricing and create authoritative order snapshot.
POST /orders/:id/payment	Initiate payment through provider adapter.
POST /payments/webhooks/:provider	Verify, deduplicate, reconcile and update state transactionally.
GET /orders/:id	Return authorized order/payment/ticket status.
POST /orders/:id/refund	Create authorized refund according to policy.
GET /organizer/settlements	Show authorized settlement records.
GET /organizer/reports	Show reconciled sales/fees/refunds/settlements.

18. Agent Implementation Rules

- Never mark an order paid from a frontend success page alone.
- Never calculate authoritative payment totals in the browser.
- Never issue a ticket from an unverified webhook.
- Never trust a QR code as proof of payment without server validation.
- Never allow cross-organizer access to orders or settlements.
- Never manually alter financial records without an audit trail.
- Never hard-code commission logic in frontend code.

- Never store provider secrets in Git.
- All payment state transitions require integration tests.
- All callbacks must be safe to replay.

19. Recommended Revenue Model

For launch, TicketUG should primarily monetize through a **transaction/platform fee** rather than requiring organizer subscriptions. This aligns Natural Intellects' revenue with successful ticket sales and lowers the barrier to entry.

- Primary revenue: configured percentage or fee per ticket/order.
- Later options: premium organizer plans, promoted events, advanced analytics and enterprise event services.
- Keep revenue streams separate in accounting/reporting.

20. Final Architecture Decision

Decision: TicketUG is a marketplace/ticketing platform integrated with a suitable payment service provider. The preferred provider model supports organizer settlement and TicketUG platform-fee economics without Natural Intellects unnecessarily becoming custodian of organizer funds.

Therefore the system must include a provider abstraction, internal financial records, immutable transaction snapshots, verified webhooks, reconciliation, refund handling, organizer settlement records and a configurable fee engine.

21. Pre-Production Gate

- Provider's Uganda eligibility confirmed.
- Commercial fees confirmed.
- Organizer settlement flow confirmed.
- TicketUG fee collection mechanism confirmed.
- Refund responsibilities confirmed.
- Webhook/idempotency behavior tested.
- Sandbox end-to-end purchase passed.
- Duplicate callback/payment tests passed.
- Reconciliation test passed.
- Production credentials securely configured.
- Legal/compliance review completed.

22. Relationship to Master Specification

This document supplements the TicketUG Master Product & Technical Specification. If a later commercial or regulatory decision changes the payment model, update the Master Specification, database design, API contract, business rules, tests and architecture decision records together.